Algoritmos de Varrimento para Desenho de Primitivas 2D

24T12 – Sala 3F5

Bruno Motta de Carvalho

DIMAp – Sala 15 – Ramal 327

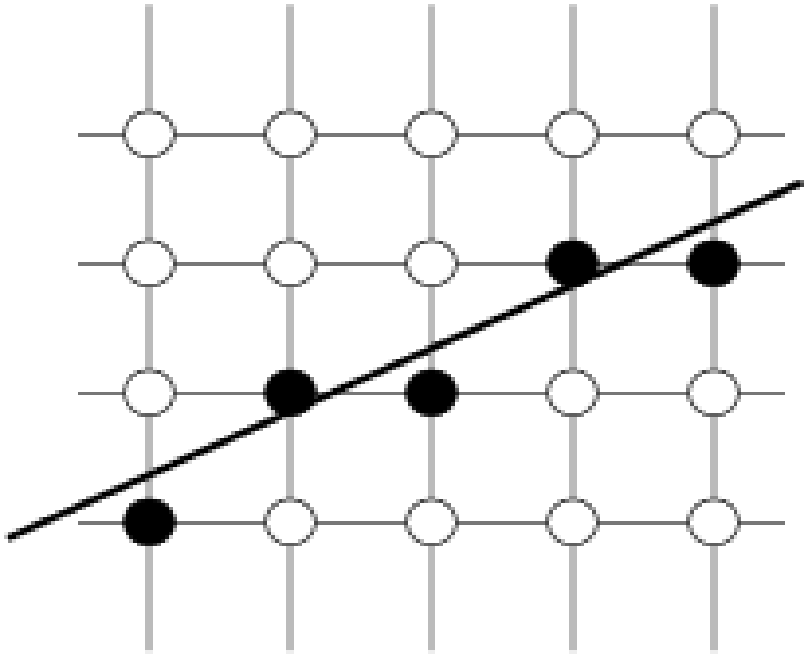
Desenhando linhas

- Sequência de pixels deve estar o mais próximo possível da linha original
- Quais propriedades uma linha deve ter?
  - 1 pixel aceso por linha ou coluna (dependendo de sua inclinação)
  - Devem ter intensidade constante, independente de sua orientação e tamanho
  - Rapidez
  - Linhas largas, estilos, pontos finais

Desenhando linhas

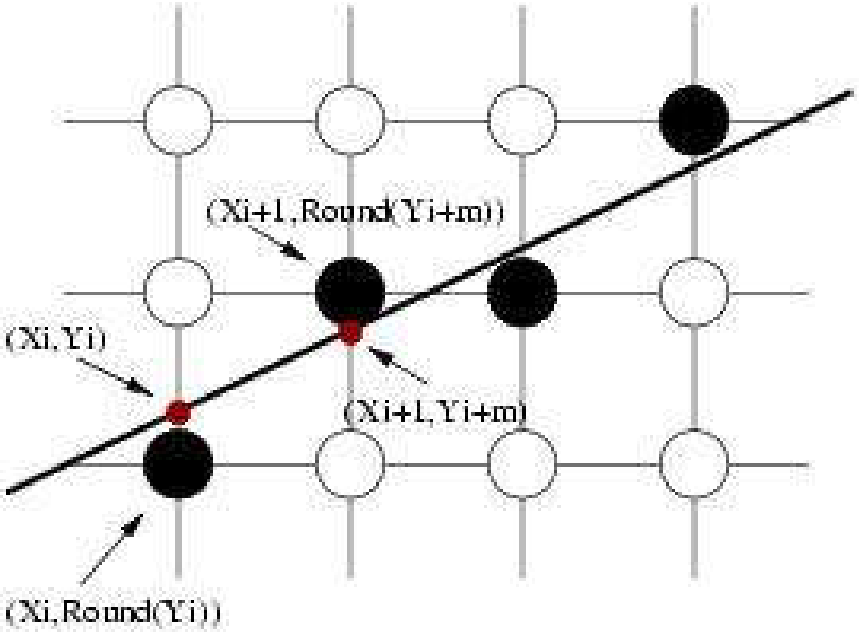
- Algoritmo incremental básico

```
int x0,y0,x1,y1,x,valor;
float dx,dy,y,m;
   dy=y1-y0;
   dx=x1-x0;
   m=dy/dx;
   y=y0;
   for(x=x0;x<=x1;x++) {
      WritePixel(x, Round(y),
                        valor);
      y+=m;
   }
```



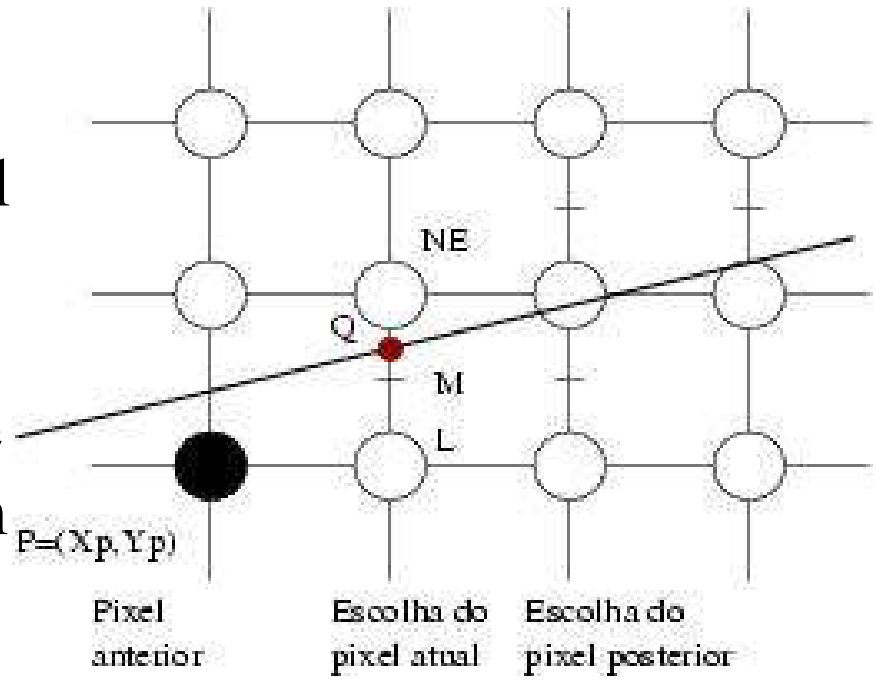
Algoritmo Incremental Básico

- Traça linhas da esquerda para a direita
- Se |m|>1, os papéis de x e y devem ser trocados
- Utiliza floats e a função Round()



Algoritmo do Ponto Médio

- Utiliza aritmética inteira
- Cálculo de (x_{i+1}, y_{i+1}) é feito de forma incremental
- Assume inclinação da linha entre 0 e 1
- Produz mesma saída que o algoritmo de Bresenham
- Em que lado da linha o ponto médio (M) está localizado?



Algoritmo do Ponto Médio

- Representando a linha pela função implícita
- $F(x,y)=ax+by+c=0$
- A equação da linha pode ser escrita como
- $y=(dy/dx)x+B, logo, F(x,y)=dy\,x-dx\,y+B\,dx=0$
- A equação acima resulta em 0 para pontos na linha, é positiva para pontos abaixo da linha e negativa para pontos acima
- Para se usar o critério do ponto médio deve-se avaliar $F(M)=F(x_p+1, y_p+1/2)=d$

Algoritmo do Ponto Médio

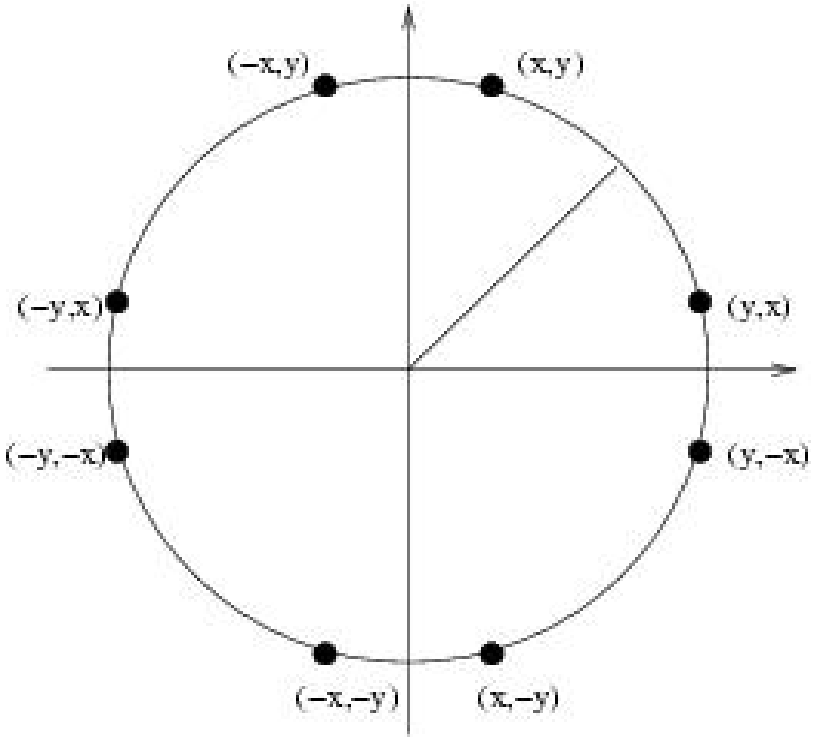
```
int x0,y0,x1,y1,valor,dx,dy,
    incrL,incrNE,d,x,y;
  dx=x1-x0;dy=y1-y0;d=2*dy-dx;
  incrL=2*dy;incrNE=2*(dy-dx);
  x=x0;y=y0;
  WritePixel(x,y,valor);
  while(x<x1) {
    if(d<=0) {
      d+=incrL;
      x++;
    }
    else {
      d+=incrNE;
      x++;
      y++;
    }
  }
  WritePixel(x,y,valor);
}
```

Algoritmo do Ponto Médio

- Ordem dos pontos inicial e final. Escolha do ponto quando linha passa exatamente no ponto médio deve ser consistente entre as duas direções
- Tratando janelas de recorte (clipping). Deve-se usar o valor real do ponto no icício da janela de recorte para inicialização do algoritmo
- Variando intensidades dos pontos em função da inclinação da linha

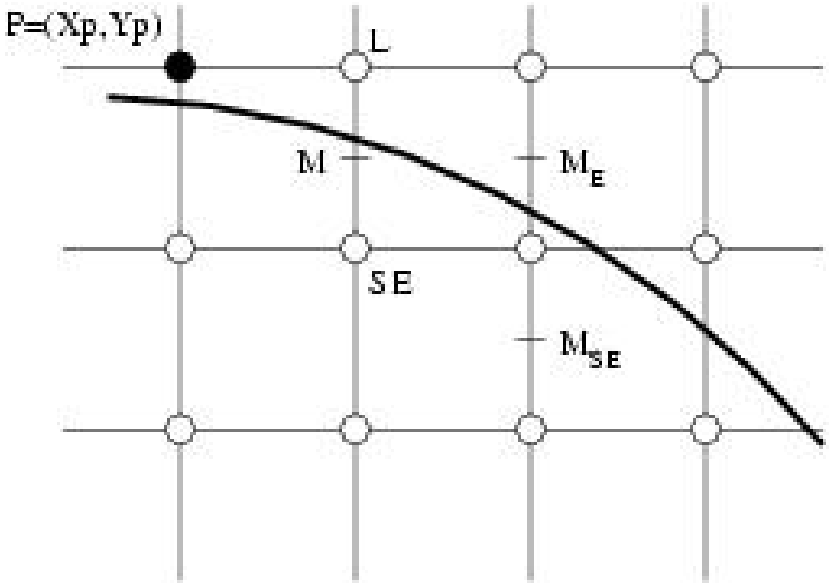
Desenhando Círculos

- 8-Simetria – coordenadas de 45º de arco do círculo podem ser replicadas gerando o círculo completo
- Algoritmo incremental básico é lento e não produz bons resultados



Algoritmo do Ponto Médio

- Considere apenas o segundo octante do círculo, de x=0 até x=y=R/sqrt(2)
- $F(x,y)=x^2 + y^2 - R^2$ é positiva for a do círculo e negativa dentro



- $d_{old}=F(x_p+1, y_p-1/2)=(x_p+1)^2+(y_p-1)^2-R^2$

Algoritmo do Ponto Médio

- Se L for escolhido, o próximo ponto médio vai ser $d_{new} = F(x_p + 2, y_p - 1/2) = (x_p + 2)^2 + (y_p - 1/2)^2 - R^2$
- e o incremento é $D_E = 2x_p + 3$
- Caso SE seja escolhido, o próximo ponto médio é $d_{new} = F(x_p + 2, y_p - 3/2) = (x_p + 2)^2 + (y_p - 3/2)^2 - R^2$
- e o incremento é $D_{SE} = 2x_p - 2y_p + 5$
- Note que as diferenças agora não são constantes. Solução: Utilizar diferenças de segunda-ordem

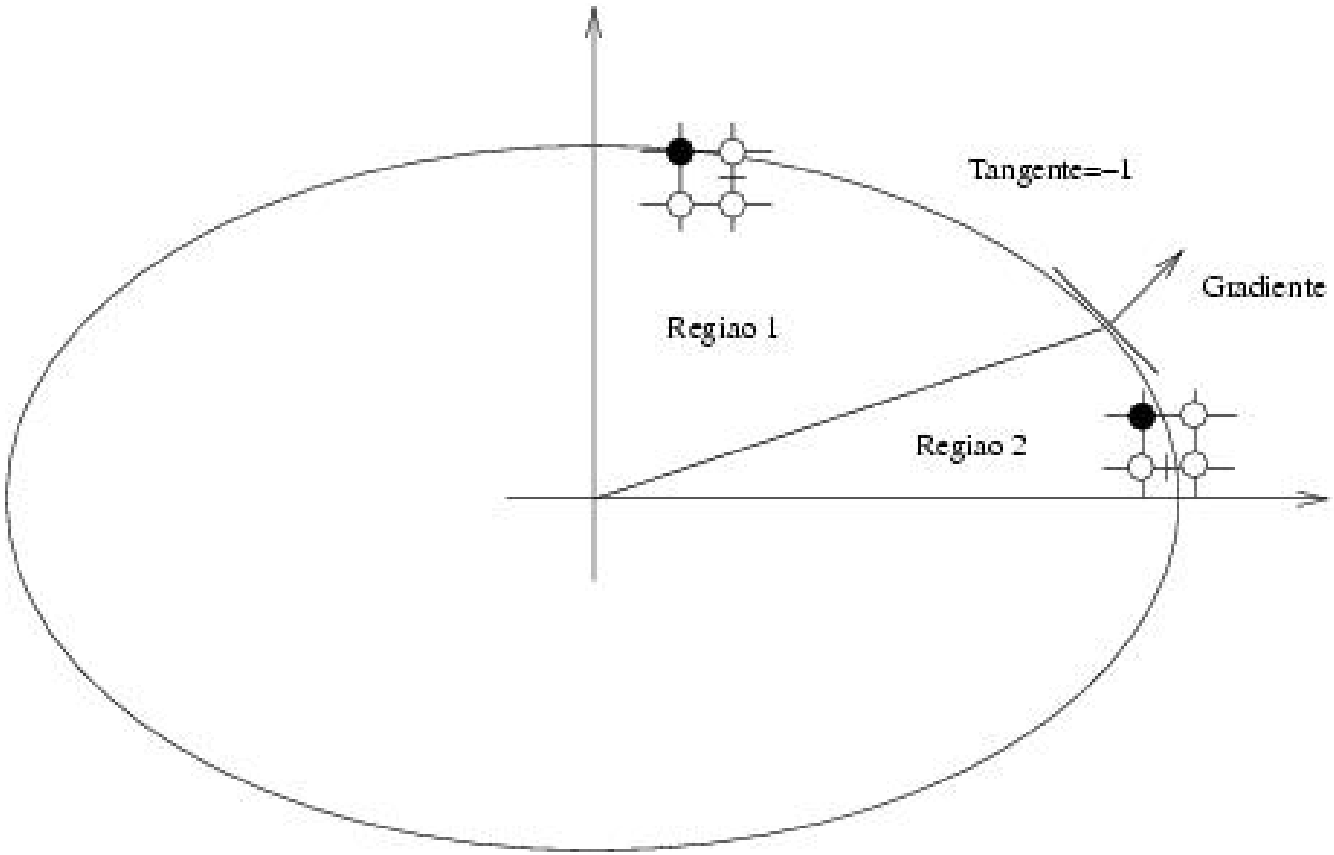
Algoritmo do Ponto Médio

```
int raio,valor,x,y,deltaL,deltaSE;
  x=0; y=raio; d=1-raio;
  CirclePoints(x,y,valor);
  while(y>x) {
    if(d>0) {
      d+=2*x+3;
      x++;
    }
   else {
     d+=2*(x-y)+5;
     x++;
     y--;
   }
   CirclePoints(x,y,valor);
}
```

Algoritmo do Ponto Médio

```
int raio,valor,x,y,deltaL,deltaSE;
  x=0; y=raio; d=1-raio;
  deltaL=3; deltaSE=2*raio+5;
  CirclePoints(x,y,valor);
  while(y>x) {
    if(d>0) {
      d+=deltaL;
      deltaL+=2;
      deltaSE+=2;
      x++;
    }
    else {
      d+=deltaSE;
      deltaL+=2;
      deltaSE+=4;
      x++;
      y--;
    }
    CirclePoints(x,y,valor);
  }
```

Desenhando Elipses



Desenhando Elipses

- A elipse é descrita por
- $F(x,y)=b^2x^2+a^2y^2-a^2b^2=0$
- centrada em (0,0)
- A mudança de regiões ocorre quando
- $a^2(y_p-1/2)\leq b^2(x_p+1)$
- Agora nós temos duas variáveis de decisão
- Pode-se utilizar a técnica de diferenças mais uma vez para acelerar a execução do algoritmo

Desenhando Elipses

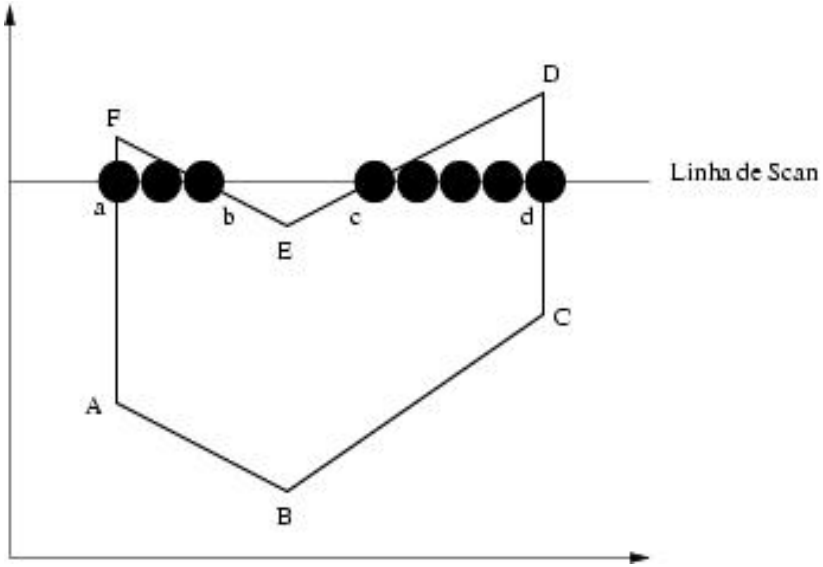
```
int a,b,valor,x,y;
float d1,d2;
  x=0; y=b; d1=b^2 -a^2b + a^2/4;
  EllipsePoints(x,y,valor);
  while(a^2(y-1/2)>b^2(x+1)) {
      if(d1<0) {
        d1+=b^2(2x+3); x++;
      }
      else {
         d1+=b^2(2x+3) + a^2(-2y+2);
         x++; y--;
      }
      EllipsePoints(x,y,valor);
}
d2=b^2(x+1/2)^2 + a^2(y-1)^2 – a^2b^2;
while(y>0) {
    if(d2<0) {
      d2+=b^2(2x+2) + a^2(-2y+3);
      x++; y--;
    }
    else {
       d2+=^2(-2y+3);
       y--;
    }
    EllipsePoints(x,y,valor);
}
```

Preenchendo Retângulos

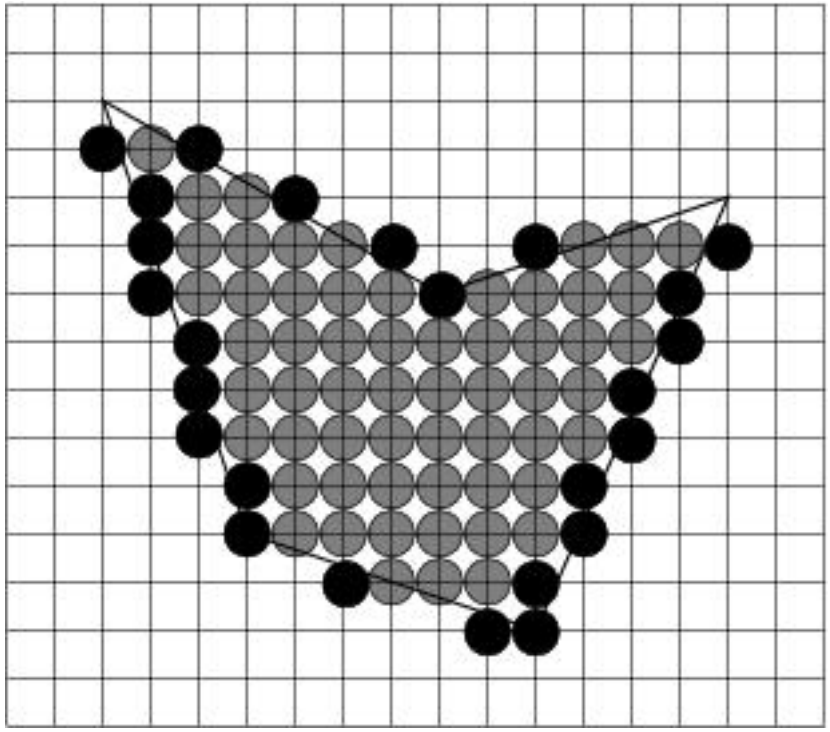
- Utiliza-se diferentes tipos de “coerência” para facilitar esta tarefa, por exemplo, espacial, de span, de linha (scan-line) e de aresta (edge)
- Escrita de pixels em bloco acelera o processo
- Problemas com bordas compartilhadas por mais de um retângulo. Solução – desenhar somente as arestas esquerda e inferior
- Quais os problemas desta solução?

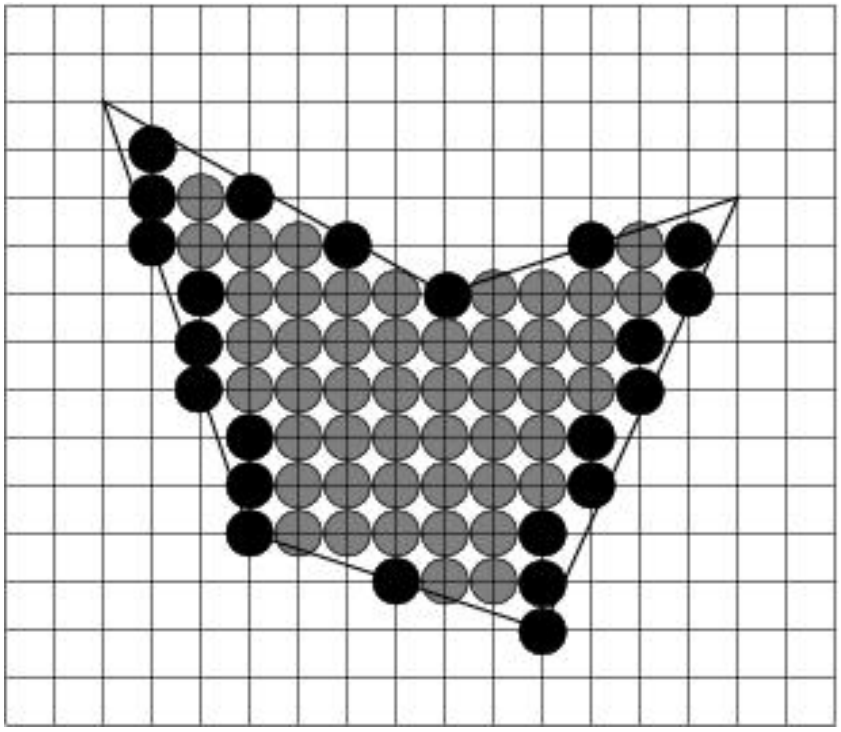
Preenchendo Polígonos

- Método funciona computando spans entre arestas à esquerda e à direita do polígono
- Métodos simples que não utiliza coerência de arestas para acelerar sua execução



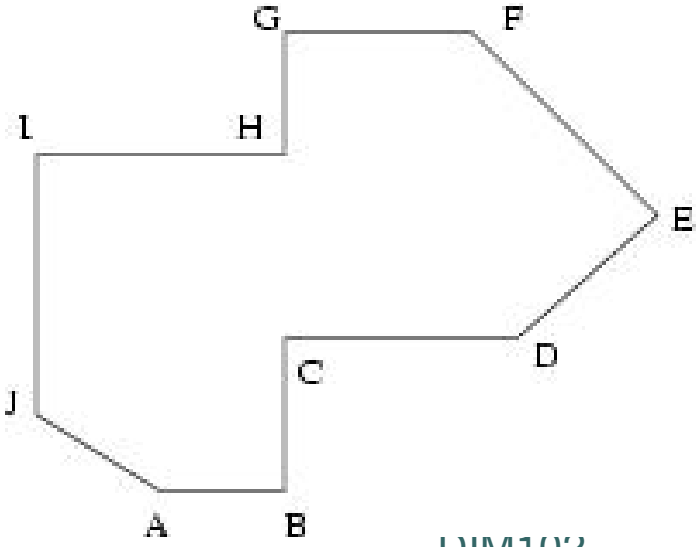
Preenchendo Polígonos

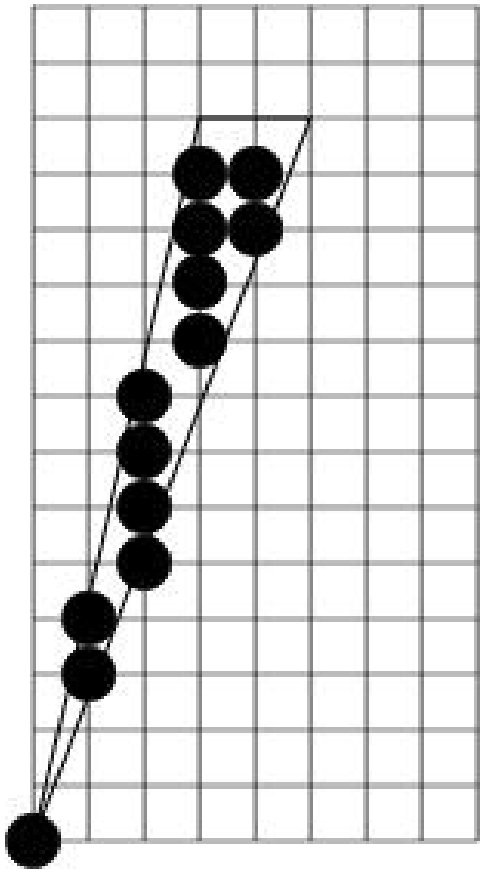




Preenchendo Polígonos

- Linhas horizontais – uso de paridade para controle dos spans
- Slivers – área poligonal fina cujo interior não contém um span para cada linha de scan





Preenchendo Polígonos

- Coerência de arestas – muitas arestas que intersectam a linha de scan i também intersectam a linha de scan i+1
- Uso de uma tabela de arestas ativas (AET) e de uma tabela de arestas (ET) global
- As arestas da AET são ordenadas pelos seus valores de interseção x. Pares destes valores (arredondados) são extremos de um span
- Uso de um algoritmo incremental para atualização das interseções a cada nova linha de scan

Preenchendo Polígonos

- Para tornar as operações sobre a AET mais eficientes, se utiliza a ET que armazena as arestas ordenadas pelas suas coordenadas y_{min}
- inclinação da linha (m) também é armazenada nas tabelas de arestas
- Para cada linha de scan, os spans são calculados e preenchidos. Depois arestas cujo valor y_{max} = y são removidas e novas arestas cujo valor y_{min} = y são adicionadas

Preenchendo com Padrões

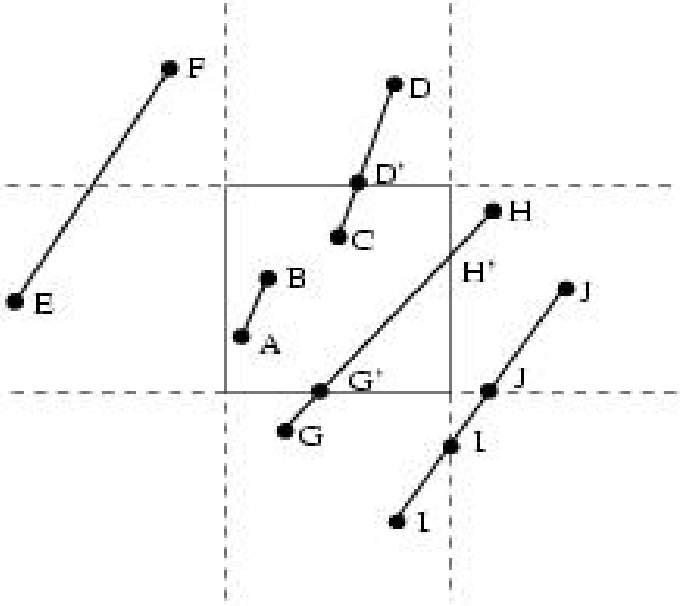
- Qual a relação da primitiva a ser preenchida com o padrão?
- Fixar um local ou vértice da primitiva para o início da textura representando o padrão
- Considerar a tela como se fosse completamente preenchida pelo padrão, mas somente visível dentro da primitiva
- Diferenças entre as duas técnicas
- Uso de escritas de pixels em bloco

Primitivas Largas

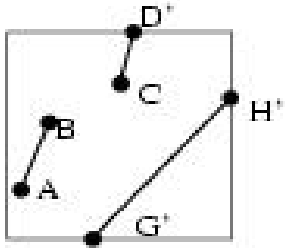
- Copiando pixels
- Canetas móveis (footprint)
- Preenchendo áreas entre bordas
- Aproximação por polilinhas largas

Recorte de Linhas em Áreas Retangulares

- Cálculo direto se os pontos finais estão dentro do retângulo
- Linhas trivialmente aceitas ou rejeitadas
- Resolvendo equações simultâneas paramétricas

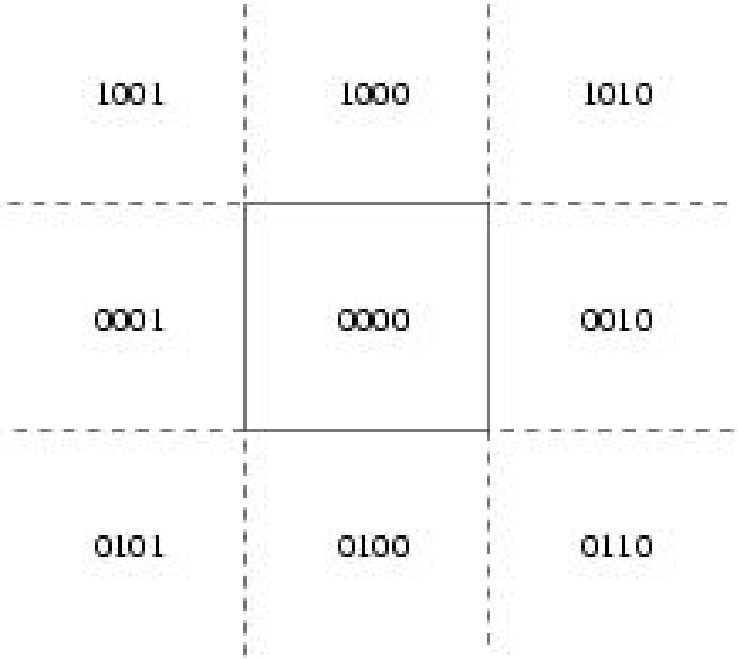


$$x = x_0 + t(x_1 - x_0)$$
$$y = y_0 + t(y_1 - y_0)$$



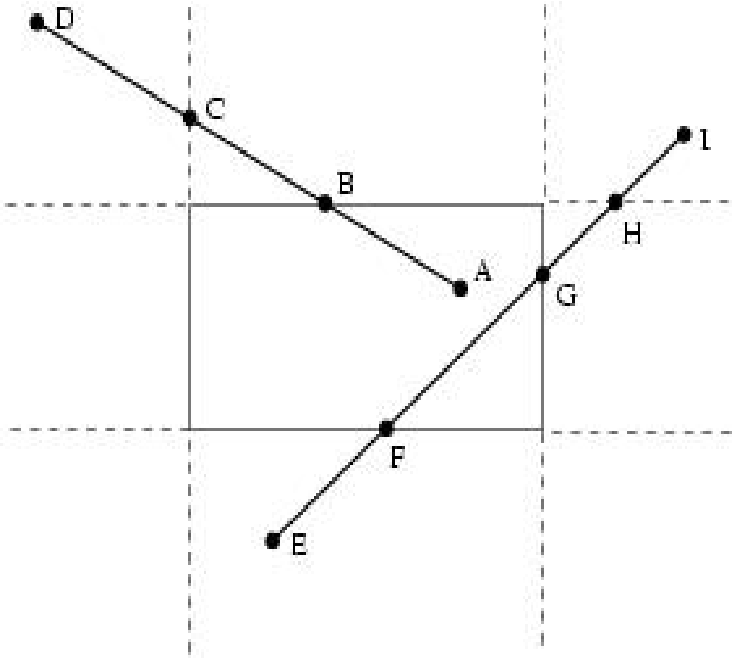
Algoritmo de Recorte de Linhas de Cohen-Sutherland

- Pontos finais são checados
- Divisão da área total em regiões
- Se o **and** lógico dos códigos dos pontos finais não é zero, a linha pode ser rejeitada “trivialmente”



Algoritmo de Recorte de Linhas de Cohen-Sutherland

- Linhas que não podem ser trivialmente aceitas ou rejeitadas são subdivididas em dois segmentos e ao menos um pode ser descartado
- Algoritmo é executado até 4 vezes por linha



Algoritmo de Recorte de Linhas Paramétrico

- Calcula o valor t na representação paramétrica da linha para o ponto que intersecta a linha de recorte

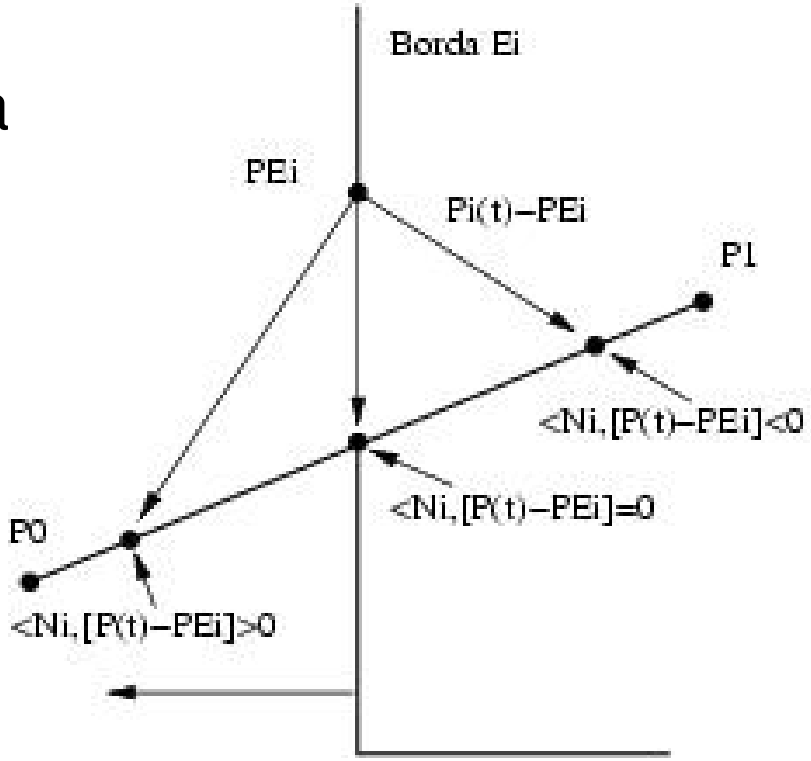
$$N_i \cdot \left[P(t) - P_{E_i} \right] = 0$$

$$N_i \cdot \left[P_0 + (P_1 - P_0)t - P_{E_i} \right] = 0$$

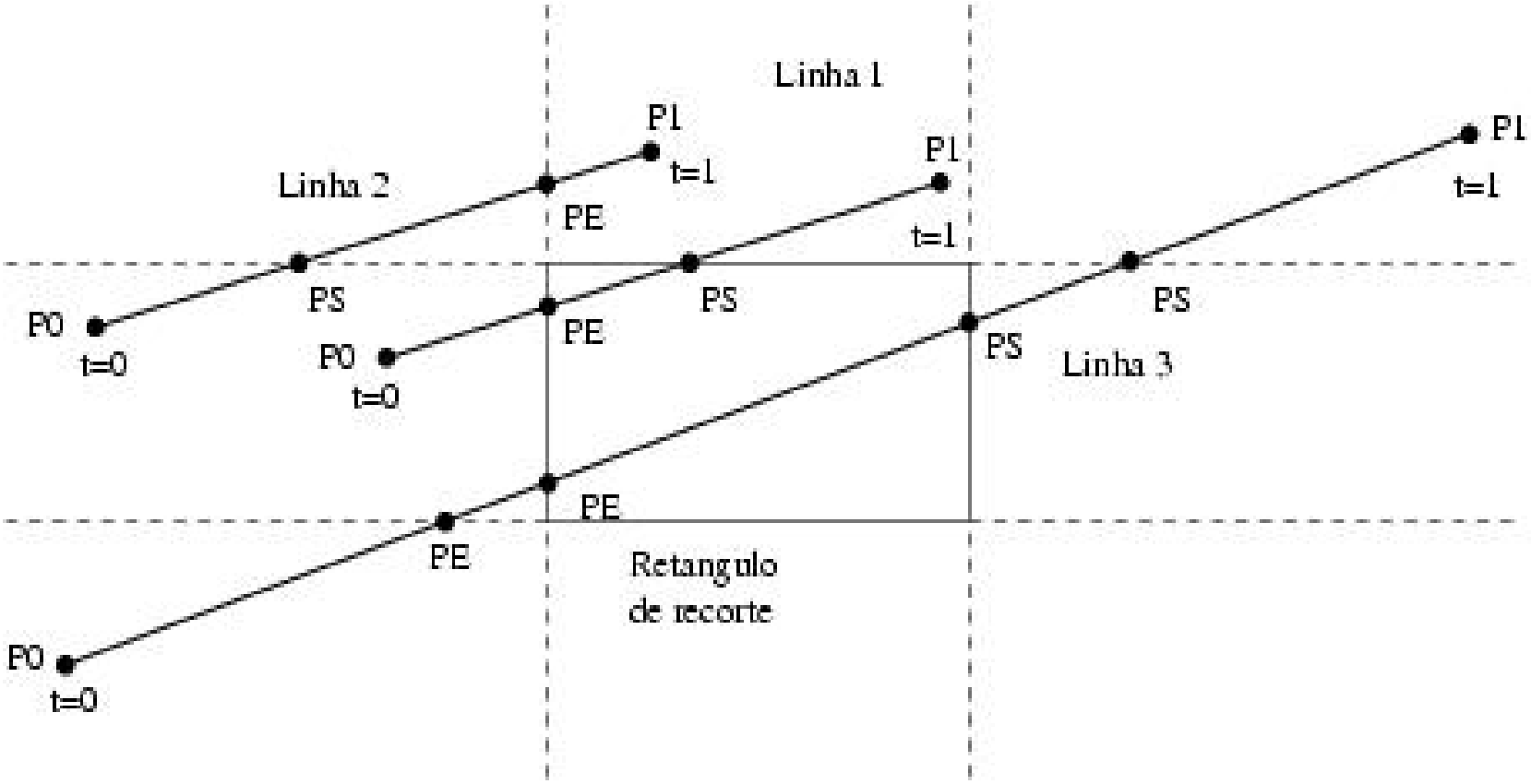
$$N_i \cdot \left[P_0 - P_{E_i} \right] + N_i \cdot \left[P_1 - P_0 \right] t = 0$$

$$t = \frac{(N_i \cdot \left[P_0 - P_{E_i} \right])}{(-N_i \cdot D)}$$

$$\text{onde } D = (P_1 - P_0)$$



Algoritmo de Recorte de Linhas Paramétrico



Algoritmo de Recorte de Linhas Paramétrico

- Para cada aresta do retângulo de recorte se calcula o valor t de interseção, descartando os valores t<0 e t>1
- As interseções são marcadas como potencialmente entrando (PE) ou potencialmente saindo (PS)

$$N_i \cdot D < 0 \rightarrow PE(\text{angulo} > 90^o)$$
$$N_i \cdot D < 0 \rightarrow PS(\text{angulo} < 90^o)$$

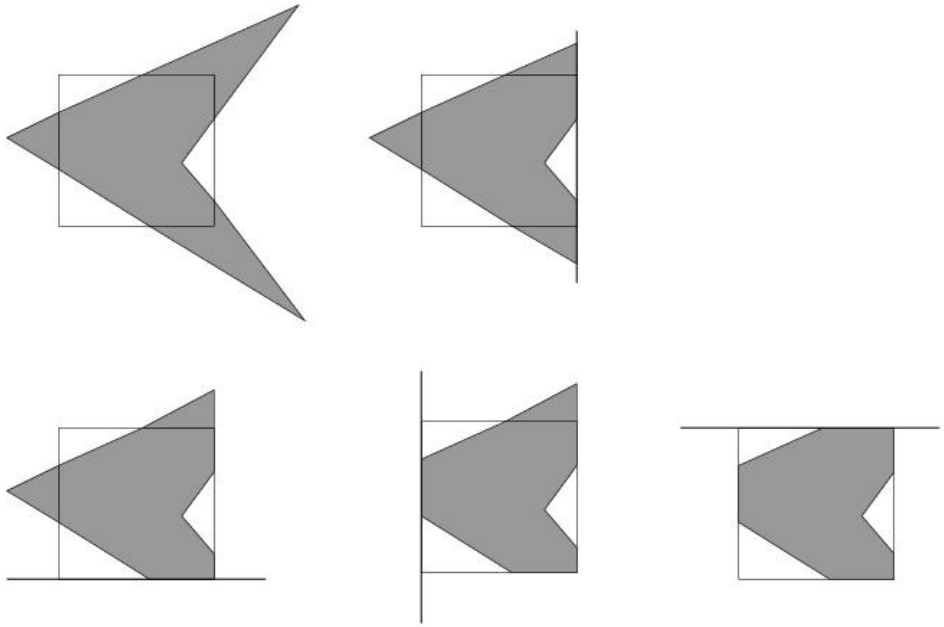
Algoritmo de Recorte de Linhas Paramétrico

```
{
 dx=x1-x0; dy=y1-y0;
 visivel=0;
 if(dx==0 && dy==0 && ClipPoint
   (x0,y0))
      visivel=1;
 else {
    tE=0;tL=1;
    if Clipt(dx,xmin-x0,tE,tS)
      if Clipt(-dx,x0-xmax,tE,tS)
        if Clipt(dy,ymin-y0,tE,tS)
          if Clipt(-dy,y0-ymax,tE,tS) {
            visivel=1;
            if(tS<1) {
              x1=x0+tS*dx;
              y1=y0+tS*dy;
            }
            if(tE>0) {
              x0=x0+tE*dx;
              y0=y0+tE*dy;
            }
          }
        }
      }
```

Recorte de Círculos e Elipses

- Círculo é testado hierarquicamente para determinar se pode ser trivialmente aceito ou rejeitado
- Subdivisão pode ir até os octantes e partir daí se calcular suas interseções analiticamente
- Elipses podem ser subdivididas até o nível de quadrantes
- Se a conversão de scan é rápida ou o círculo ou elipse são pequenos, pode ser vantajoso testar os pixels de borda individualmente

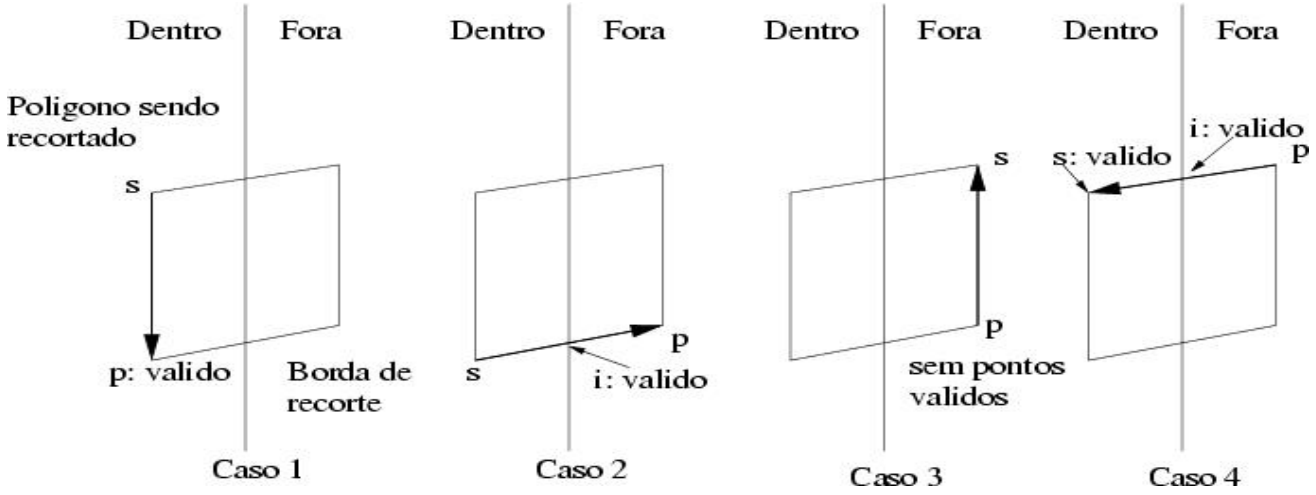
Alg. de Rec. de Políg. de Sutherland-Hodgman



- Utiliza a estratégia dividir-e-conquistar
- Mais geral, pode ser utilizado para recorte de um polígono convexo ou côncavo contra um polígono de recorte convexo
- O recorte é efetuado aresta por aresta do polígono de recorte

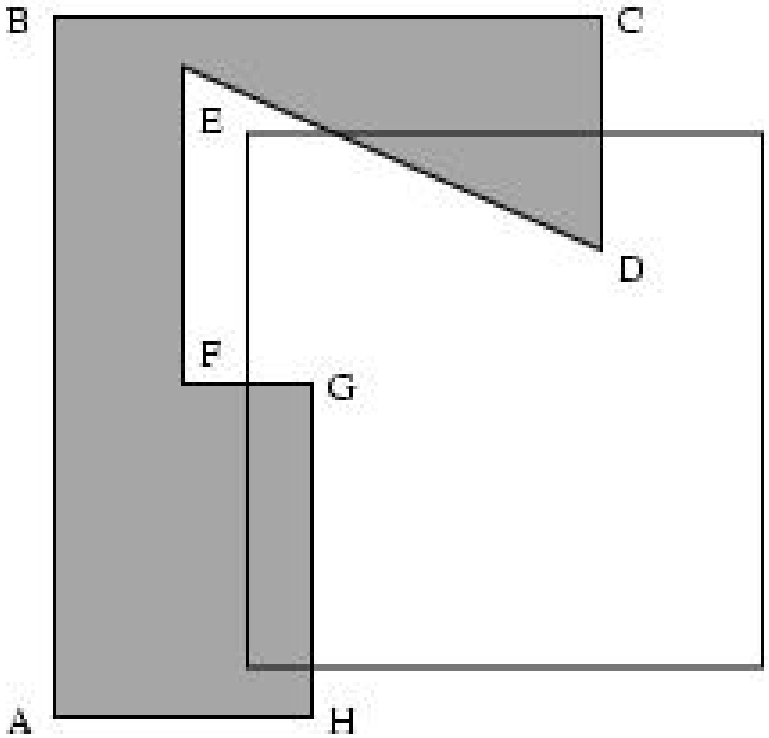
Alg. de Rec. de Políg. de Sutherland-Hodgman

- Vértices são adicionados ao polígono recortado de acordo com as regras abaixo
- Pode ser implementado como um pipeline de recortes. Vantajoso em uma implementação em hardware



Alg. de Rec. de Políg. de Sutherland-Hodgman

- Arestas falsas podem ser incluídas pelo algoritmo
- Pós-processamento é utilizado para remover estas arestas falsas



Gerando Letras

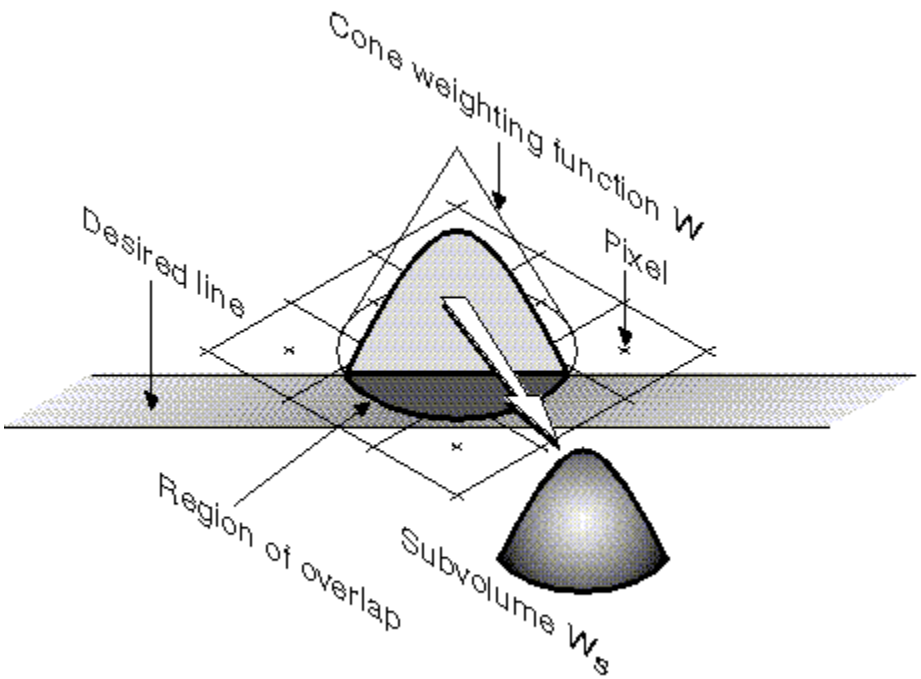
- Podem ser definidas como bitmaps ou como curvas ou polígonos
- A primeira opção implica no uso de uma cache de fontes, de onde são copiadas letras para o frame-buffer
- Memória necessária aumenta rapidamente
- A segunda opção geralmente usa uma única descrição abstrata de cada letra
- Transformações podem ser aplicadas às letras facilmente

Antialiasing

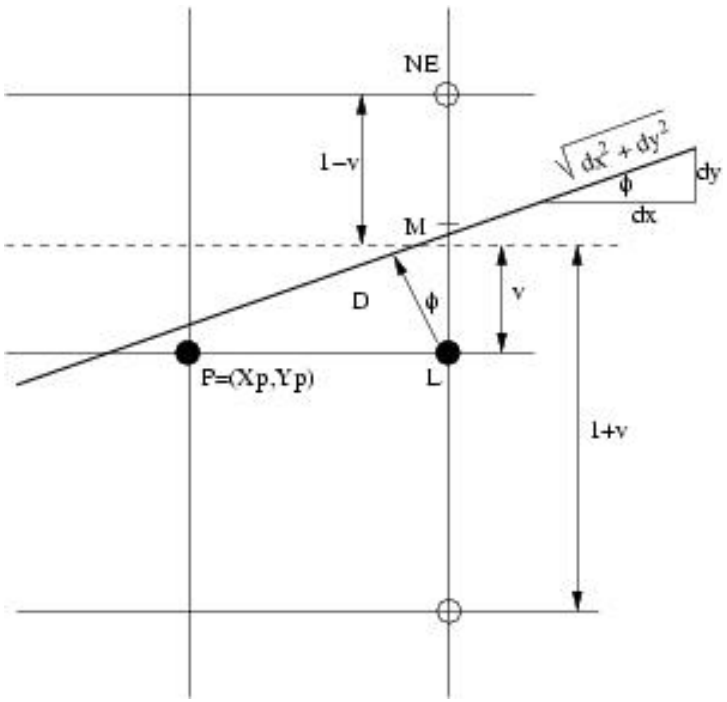
- Aumento de resolução – Solução cara que alivia mas não soluciona o problema
- Amostragem por área sem peso – Considerar que linhas têm uma largura associada e utilizar as áreas de interseção no cálculo da intensidade a ser desenhada
  - Intensidade do pixel diminui com a distância para a linha
  - Pixels não interceptados não são afetados
  - Áreas iguais contribuem intensidades iguais

Amostragem por área

- Amostragem por área com peso – Similar a anterior, porém utiliza filtros onde a eas mais próximas ao pixel contribuem mais que áreas mais afastadas



Linhas Antialiased de Gupta-Sproull



- Pré-calcula o subvolume de um filtro normalizado à distâncias diferentes do centro do pixel e armazena em uma tabela (LUT)
- Algoritmo do ponto médio pode ser modificado para gerar linhas antialiased
- LUT funciona para linhas de uma largura somente